

Dokumentation - NeuralNetworkPipeline

Martin Dutsch

Einleitung

Dieses Projekt entstand aus dem Wunsch heraus, eine generalisierte Pipeline für neuronale Netzwerke aus dem Framework Keras (TensorFlow) zu realisieren, die funktional der Pipeline für klassische ML-Modelle aus der Bibliothek scikit-learn ähnelt. Eine solche Pipeline bietet ein kompaktes Design, bei dem viele Daten-Vorverarbeitungsschritte sowie der eigentliche Trainingsprozess in wenigen Befehlen gebündelt werden. Dabei soll die Pipeline vollständige Autarkie von den Trainingsdaten gewährleisten und somit Vorhersagen mit Testdaten ohne Zugriff auf die ursprünglichen Trainingsdaten realisieren.

Inhaltsverzeichnis

1. Implementierung	3
1.1. Architektur	3
1.2. Datenvorverarbeitung	5
2. Ausführung und Ergebnisse	12

1. Implementierung

Die Pipeline ist als objektorientierte Klasse mit dem Namen **NeuralNetworkPipeline** umgesetzt. Dabei greifen die Methoden auf eigens entwickelte Bibliotheken mit modularen Funktionen zu. So sind alle Funktionen für die reine Datenvorverarbeitung in der Bibliothek **DataPreprocessingFunctions** sowie die Funktionen zur Visualisierung in der Bibliothek **DisplayFunctions** implementiert.

1.1. Architektur

Die Konzeption der Methoden und Attribute der Pipeline zielt auf eine möglichst geringe Anzahl von Methodenaufrufen für die Vorbereitung und Durchführung des Trainings ab. Damit dabei die Anzahl der Parameter pro Methode nicht zu groß und somit benutzerunfreundlich wird, verwendet die Pipeline interne Attribute zur Reduktion der Übergabeparameter.

Ein Teil der Attribute repräsentiert die Optionen für den Trainingsalgorithmus sowie die Konfiguration für die Datenvorverarbeitung als notwendige Voreinstellungen für das Training.

Als Optionen für den Trainingsalgorithmus lassen sich der Optimierer, die zu minimierende Verlustfunktion sowie gegebenenfalls zusätzliche Bewertungskriterien für die Modellleistung festlegen. Über die Parameter der Methode **set_trainalgo** definiert man diese drei Optionen ein und weist sie den entsprechenden Attributen in der Pipeline zu.

Mit den Parametern der Methode **set_preprocessing_options** definiert der Benutzer die Konfiguration für die Datenvorverarbeitung und setzt damit die entsprechenden Attribute. Die Konfiguration für die Datenvorverarbeitung definiert die individuelle Vorverarbeitung der Trainingsdaten. Dabei können Features für einzelne Vorverarbeitungsschritte ausgewählt bzw. ausgeschlossen sowie spezifische Parameter für bestimmte Vorverarbeitungsschritte festgelegt werden. Eine detaillierte Erklärung zu den einzelnen Parametern der Konfiguration für die Datenvorverarbeitung lässt sich im Unterkapitel Datenvorverarbeitung finden.

Weitere Attribute der Pipeline repräsentieren die Transformation States und werden durch die Trainingsmethode **train** gesetzt. Die Transformation States bestimmen die Vorverarbeitung der Validierungs- und Testdaten in Abhängigkeit von der Vorverarbeitung der Trainingsdaten. Sie werden ausschließlich aus der Vorverarbeitung der Trainingsdaten abgeleitet und verhindern dadurch Data Leakage zwischen den Trainingsdaten und den Validierungs- bzw. Testdaten. Dadurch durchlaufen die Validierungs- und Testdaten die gleichen Transformationen wie die

Trainingsdaten und werden analog zu diesen vorverarbeitet. Zu den Transformation States gehören beispielsweise die anhand der Trainingsdaten berechneten Imputed Values sowie die trainierten Skalierer und Encoder. Eine detaillierte Erläuterung zu den einzelnen States findet sich im Unterkapitel Datenvorverarbeitung. Durch die Speicherung der Transformation States in den Attributen kann die Pipeline Vorhersagen mit den Testdaten erstellen, ohne dabei auf die Trainingsdaten zugreifen zu müssen. Somit bilden die Attribute zur Repräsentation der Transformation States einen wichtigen Baustein für die Autarkie der Pipeline.

Die Methode **train** führt das Training inklusive der Vorverarbeitung der Trainings- und Validierungsdaten sowie die Darstellung des Trainingsverlaufes aus. Als Parameter für diese Methode sind die Daten als `pandas.DataFrame`, die Split-Ratio, Batchsize und Epochs für das Training sowie gegebenenfalls eine Early-Stopping-Strategy anzugeben. Bei der Ausführung von **train** werden zuerst die Daten anhand der angegebenen Split-Ratio in Trainings- und Validierungsdaten aufgeteilt. Anschließend verarbeitet die Methode die Trainingsdaten anhand der Konfiguration für die Datenvorverarbeitung vor. Aus dieser Vorverarbeitung der Trainingsdaten generiert die Methode die Transformation States und weist diese den entsprechenden Attributen zu. Danach erfolgt die Vorverarbeitung der Validierungsdaten anhand der Transformation States. Die Trainings- und Validierungsdaten sind nun für das Training des neuronalen Netzwerkes nutzbar. Basierend auf den Optionen für den Trainingsalgorithmus trainiert die Methode dann das neuronale Netzwerk mit den vorverarbeiteten Trainings- und Validierungsdaten. Anschließend wird der Verlauf des Trainings in einem Diagramm dargestellt und in dem entsprechenden Attribut der Pipeline gespeichert.

Zusätzlich zu den Attributen für die Transformation States bilden das Attribut zur Speicherung des zu trainierenden neuronalen Netzwerkes und die Methode **save_pipeline_in_joblib** die beiden weiteren wichtigen Bausteine für die Autarkie der Pipeline. Mit der Methode **set_model** ist das zu trainierende neuronale Netzwerk im entsprechenden Attribut der Pipeline für das Training zu hinterlegen. Mit der Methode **save_pipeline_in_joblib** lässt sich die gesamte Pipeline als Joblib-Datei exportieren.

Somit lässt sich nach dem Training die Pipeline, inklusive des trainierten neuronalen Netzwerkes und der Transformation States, abspeichern und an einem beliebigen Ort für Vorhersagen laden. Durch die in den Attributen der Pipeline abgespeicherten Transformation States sowie das dort ebenfalls hinterlegte neuronale Netzwerk ist kein Zugriff auf die Trainingsdaten für die Vorhersage mit Testdaten nötig.

Darüber hinaus existiert die Hilfsmethode **show_required_input_shape** zur Ausgabe der Dimension (Shape) der vorverarbeiteten Trainingsdaten, um die korrekte Größe der Eingangsschicht (Input Layer) des zu trainierenden neuronalen Netzwerkes festlegen zu können. Die Form der vorverarbeiteten Trainingsdaten wird in Abhängigkeit von den Attributen der festgelegten Konfiguration für die Datenvorverarbeitung ermittelt.

Vorhersagen mit den Testdaten lassen sich mit der Methode **predict** erstellen. Dafür müssen die Attribute für die Transformation States gesetzt sowie ein neuronales Netzwerk im entsprechenden Attribut hinterlegt sein. Da die Attribute für die Transformation States ausschließlich durch die Trainingsmethode gesetzt werden können, sind Vorhersagen ohne vorheriges Training oder Laden einer gespeicherten Pipeline nicht möglich.

1.2. Datenvorverarbeitung

Der Hauptteil der Pipeline besteht aus der Datenvorverarbeitung. Sie erfolgt bei jedem Training für die Trainings- und Validierungsdaten sowie bei der Vorhersage für die Testdaten. Dabei werden aus der Vorverarbeitung der Trainingsdaten die Transformation States und somit die Vorgaben für die Vorverarbeitung der Validierungs- und Testdaten abgeleitet.

Die Datenvorverarbeitung der Trainingsdaten erfolgt in diesen Schritten:

1. Datentyperkennung der Features
2. Entfernung von Features
3. Imputation der Features
4. Feature Engineering

Dabei lassen sich bestimmte Features von einzelnen Vorverarbeitungsschritten über die Konfiguration für die Datenvorverarbeitung ausschließen. Im Gegensatz dazu sind die Features für das Feature Engineering explizit über die Konfiguration für die Datenvorverarbeitung anzugeben. Dies geschieht hier über ausgewählten Parameter der Methode **set_preprocessing_options**, welche die Werte dadurch gleichzeitig in den entsprechenden Pipeline-Attributen speichert.

Um Features von bestimmten Vorverarbeitungsschritten auszuschließen existieren folgende Parameter:

- **detection_excep** für den Ausschluss von Features bei der Datentyperkennung
- **drop_excep** für den Ausschluss von Features bei der automatischen Entfernung von Features
- **impute_excep** für den Ausschluss von Features bei der Imputation

Für die Auswahl von Features zur Bearbeitung mit Feature-Engineering-Prozessen gibt man die ausgewählten Features explizit mit den folgenden Parametern an:

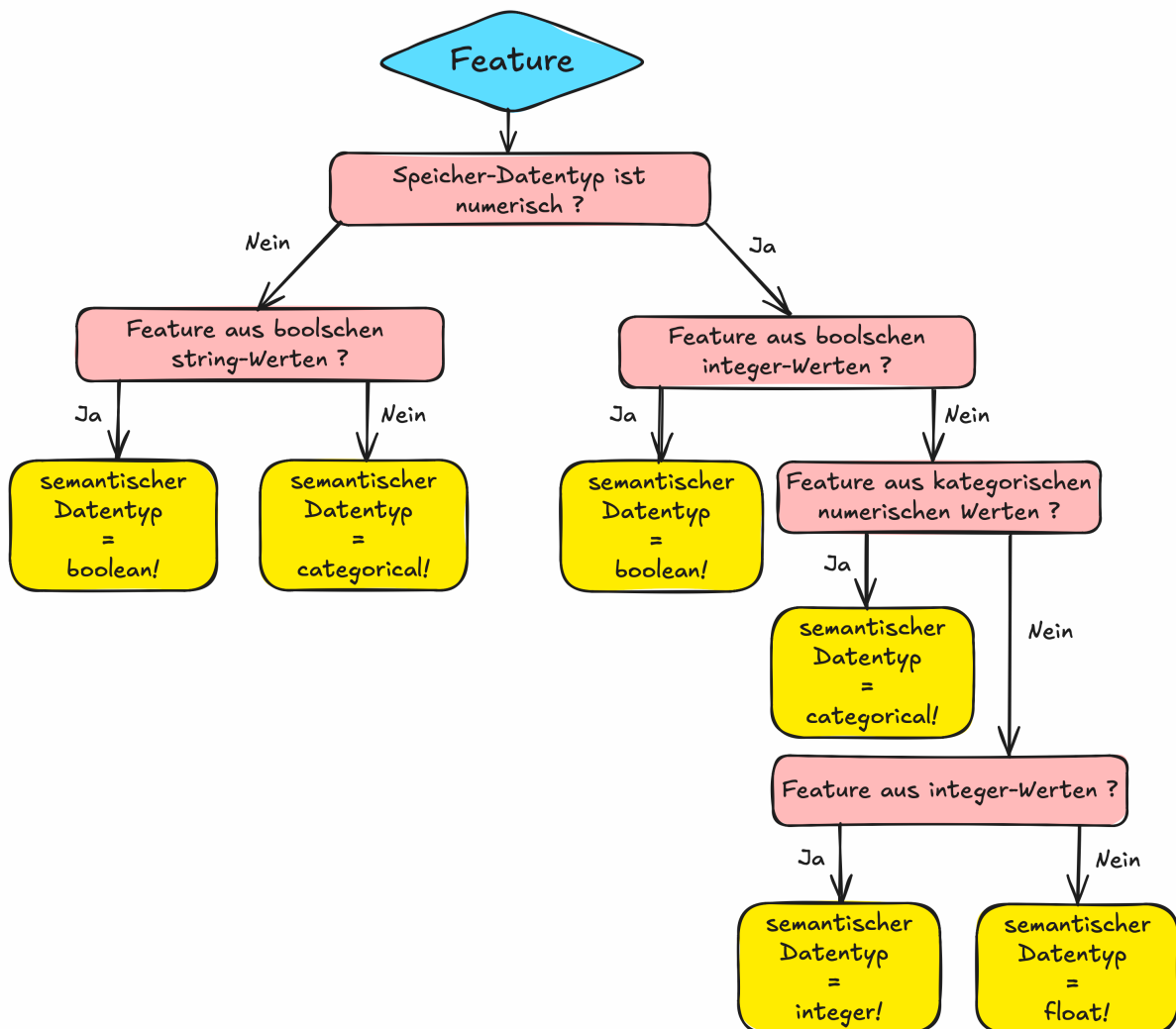
- **cols_for_onehot** für die Auswahl von Features zur Kodierung mittels One-Hot-Encoding
- **cols_and_scalers** für die Auswahl von Features zur Skalierung

Darüber hinaus existiert der Parameter **manual_columns_to_drop** für die Auswahl von Features zur manuellen Entfernung in der Datenvorverarbeitung. Bei jedem der oben genannten Parameter, außer **cols_and_scalers**, sind die jeweils ausgewählten Features mit ihrem Namen als String innerhalb einer Liste anzugeben. Beim Parameter **cols_and_scalers** wird ein Dictionary verwendet, wobei der Key den Namen und der Value die gewünschte Skalierung des Features definiert (beides ebenfalls im String-Format).

Das Herzstück der Datenvorverarbeitung ist die Datentyperkennung der Features. Diese soll aus dem Speicher-Datentyp der Features den semantischen Datentyp ableiten.

Ein anschauliches Beispiel hierfür ist ein Feature mit Missing Values, welches ansonsten nur Integer-Werte enthält. Dieses Feature wird aufgrund der Missing Values oft mit dem Speicher-Datentyp Float eingelesen, obwohl der semantische Datentyp eigentlich Integer ist.

Die Datentyperkennung ermittelt für die weiteren Vorverarbeitungsschritte den semantischen Datentyp jedes Features. Dabei läuft die Datentyperkennung systematisch nach diesem Schema ab:



Die Datentyperkennung geht von einem Speicher-Datentyp des Features aus, welcher entweder numerisch oder ein String ist. Zuerst überprüft die Datentyperkennung, ob der Speicher-Datentyp des Features numerisch ist. Ist dies nicht der Fall, wird das Feature implizit als Speicher-Datentyp String behandelt.

Bei einem nicht-numerischen Speicher-Datentyp geht man von einem Feature mit String-Werten als Einträgen aus. Daraus abgeleitet ist der semantische Datentyp des Features entweder Boolean oder Categorical. Es liegt also ein boolesches oder kategorisches Feature vor. Für diese Unterscheidung prüft die Datentyperkennung die Unique Values des Features auf boolesche String-Werte, wie beispielsweise {'true', 'false'} oder {'wahr', 'falsch'}. Bei einem Vorliegen von booleschen String-Werten als Unique Values wird der semantische Datentyp als Boolean bestimmt, andernfalls als Categorical.

Besitzt das Feature, im Gegensatz zum vorherigen Absatz, einen numerischen Speicher-Datentyp so ergeben sich daraus vier Fälle für den semantischen Datentyp. Demnach können sich Boolean, Categorical, Integer oder Float als semantischer Datentyp hinter einem numerischen Speicher-Datentyp verbergen. Der semantische Datentyp ist Boolean, wenn das Feature ausschließlich boolesche numerische Werte wie {1, 0} oder {1.0, 0.0} als Unique Values besitzt. Andernfalls wird das Features auf den semantischen Datentyp Categorical geprüft. Liegt die Cardinality des Features unter einem Schwellenwert und ist gleichzeitig kleiner als 80 % des Total Count des Features, so ist der semantische Datentyp Categorical. Dabei legt man den Schwellenwert als Option für die Datenvorverarbeitung mit dem Parameter **maxcardhiddencat** der Methode **set_preprocessing_options** fest. Lassen sich Boolean und Categorical als semantische Datentypen ausschließen, so ergeben sich noch Integer und Float als mögliche semantische Datentypen für das Feature. Der semantische Datentyp des Features ist Integer, wenn alle Einträge ohne Missing Values aus Integer und/oder ganzzahligen Fließkommazahlen bestehen. Ansonsten ist der semantische Datentyp des Features Float.

Als weitere Informationen zu den semantischen Datentypen der Features werden deren Missing-Rate und Kardinalität für die nachfolgenden Datenvorverarbeitungsschritte bestimmt.

Bei der Vorverarbeitung der Validierungs- und Testdaten erfolgt keine erneute Datentyperkennung der Features. Der in den Trainingsdaten erkannte semantische Datentyp eines Features wird als semantischer Datentyp für das entsprechende Feature in den Validierungs- und Testdaten verwendet, da dieser in den Transformation States gespeichert ist.

Die automatisierte Entfernung von Features ist der zweite Schritt der Datenvorverarbeitung. Dabei erfolgt das automatisierte Entfernen von Features anhand ihrer Kennzahlen wie der Missing-Rate und der Cardinality. Prinzipiell entfernt dieser Vorverarbeitungsschritt Features, deren Missing-Rate einen gewählten Schwellenwert überschreitet. Darüber hinaus entfernt man hier Features mit dem semantischen Datentyp Categorical, falls ihre Cardinality über einem gewählten Schwellenwert liegt. Die Schwellenwerte **maxmisspercent** für die Missing-Rate sowie **maxcardcat** für die Cardinality lassen sich als Konfiguration für die Datenvorverarbeitung über die Parameter der Methode **set_preprocessing_options** festlegen.

Für die Vorverarbeitung der Validierungs- und Testdaten werden keine neuen Entscheidungen für die automatische Entfernung von Features getroffen. Es werden die gleichen Features wie in den Trainingsdaten entfernt. Dafür sind die anhand der Trainingsdaten entfernten Features mit ihren Namen in den Transformation States gespeichert.

Die Imputation als dritten Schritt der Datenvorverarbeitung ersetzt die Missing Values der Features mit den ermittelten Imputed Values in Abhängigkeit ihres semantischen Datentyps. Außerdem erfolgt hier die Konvertierung der Features in ihren semantischen Datentyp. Ist der semantische Datentyp numerisch entspricht der Imputed Value dem Mittelwert aus den Trainingsdaten des Features. Liegt Boolean als semantischer Datentyp vor, verwendet man als Imputed Value den ermittelten Modalwert aus den Trainingsdaten. Der String-Wert 'was_missing' wird als Imputed Value beim semantischen Datentyp Categorical verwendet und zusätzlich in den Unique Values des Features ergänzt. Prinzipiell entsteht bei der Imputation eines Missing Values eine Indikator-Spalte, welche alle imputierten Datenpunkte im jeweiligen Feature markiert. Diese Indikator-Spalte wird dann als zusätzliches Feature mit dem Namen 'Feature_was_missing' verwendet.

Die Imputation der Missing Values in den Validierungs- und Testdaten erfolgt für die jeweiligen Features mit den Imputed Values, welche aus den Trainingsdaten abgeleitet sind. Diese Imputed Values sind für jedes einzelne Feature in den Transformation States abgespeichert. Für die Erstellung der Indikator-Spalten zur Markierung der Imputed Values gilt dabei folgende Konvention: Die Indikator-Spalte für das jeweilige Feature in den Validierungs- und Testdaten ist zu erstellen, wenn eine Indikator-Spalte für dieses Feature in den Trainingsdaten erstellt wurde. Dies geschieht auch, wenn das jeweilige Feature in den Validierungs- und Testdaten keine Missing Values aufweist und daher keine Imputation stattfindet. Existiert keine Indikator-Spalte für das jeweilige Feature in den Trainingsdaten, so wird auch keine Indikator-Spalte für dieses Feature in den Validierungs- und Testdaten erstellt. Sollte dann das jeweilige Feature in den Validierungs- und Testdaten Missing Values aufweisen, so erfolgt nur eine Imputation. Dadurch wird das Aufblähen der Dimension des Datensatzes verhindert. Müssten alle imputierten Datenpunkte in jedem Feature mit einer Indikator-Spalte markiert werden, so müsste für jedes Feature vorsichtshalber eine Indikator-Spalte über den gesamten Datensatz erstellt werden, da man von der Vollständigkeit eines Features in den Trainingsdaten nicht auf die Vollständigkeit in den Validierungs- und Testdaten schließen kann. Für vollständige Features über den gesamten Datensatz würden dann Indikator-Spalten erstellt werden, welche nur aus 0-Einträgen bestehen. Diese Indikator-Spalten erhöhen dann die Dimension des gesamten Datensatzes und verringern gleichzeitig die Erlernbarkeit des Datensatzes durch ein neuronales Netzwerk.

Als letzter Schritt der Datenvorverarbeitung erfolgt das Feature Engineering. Dazu gehören das One-Hot-Encoding für ausgewählte Features mit dem semantischen Datentyp Categorical und die Skalierung für ausgewählte Features mit einem numerischen semantischen Datentyp. Beim One-Hot-Encoding werden die Unique Values der Trainingsdaten des jeweiligen Features um den Wert 'unknown' erweitert. Dieser gilt als Platzhalter für unbekannte Kategoriewerte in den Unique Values der Validierungs- und Testdaten des jeweiligen Features. Für jeden Kategoriewert in den erweiterten Unique Values des Features wird eine Indikator-Spalte erstellt. Diese Indikator-Spalten ersetzen dann das jeweilige Feature in den Trainingsdaten. Bei der Skalierung für ausgewählte Features lässt sich zwischen Min-Max- und Standard-Skalierung wählen. Die Skalierungsparameter der gewählten Skalierung werden aus den Trainingsdaten des jeweiligen Features berechnet.

Das Feature Engineering der Validierungs- und Testdaten erfolgt anhand der abgeleiteten Informationen aus den Trainingsdaten. Diese Informationen sind in den Transformation States gespeichert. Beim One-Hot-Encoding ersetzt man die Unique Values des jeweiligen Features in den Validierungs- und Testdaten mit den erweiterten Unique Values des Features aus den Trainingsdaten. Dabei transformieren sich die Werte, welche nur in den Validierungs- oder Testdaten vorkommen, in den Platzhalter-Wert 'unknown'. Jetzt lässt sich das One-Hot-Encoding mit einem One-Hot-Encoder, welcher mit den Trainingsdaten des jeweiligen Features trainiert wurde, problemlos für die Validierungs- und Testdaten des jeweiligen Features durchführen. Die Skalierung des jeweiligen Features in den Validierungs- und Testdaten erfolgt hier mit dem Skalierer, welcher mit den Trainingsdaten des jeweiligen Features trainiert wurde. So wird die Skalierung anhand der Skalierungsparameter aus den Trainingsdaten für die Validierungs- und Testdaten durchgeführt.

Zusammenfassend setzt sich die Konfiguration für die Datenvorverarbeitung, welche der Benutzer über die Parameter der Methode **set_preprocessing_options** in den entsprechenden Attributen der Pipeline setzt, aus folgenden Angaben zusammen:

- **detection_excep, drop_excep** und **impute_excep**: Listen mit den Feature-Namen, die von der Datentyperkennung, der automatischen Feature-Entfernung oder der Imputation ausgeschlossen werden sollen.
- **cols_for_onehot**: Liste mit den Namen jener Features, die mittels One-Hot-Encoding transformiert werden.
- **cols_and_scalers**: Ein Dictionary, das Feature-Namen (Keys) und die gewünschte Skalierungsmethode (Values) zur Skalierung der jeweiligen Features verknüpft.
- **manual_columns_to_drop**: Liste mit den Namen der Features, die manuell entfernt werden.
- **maxcardhiddencat** (Integer): Obergrenze für die Cardinality, um Features mit einem numerische Speicher-Datentypen als semantischen Datentyp Categorical zu identifizieren.
- **maxmisspercent** und **maxcard** (Integer): Schwellenwerte für die Missing-Rate und die Cardinality (nur bei Features mit dem semantischen Datentyp Categorical) für das automatische Entfernen von Features.

Durch die Vorverarbeitung der Trainingsdaten auf Basis dieser Konfiguration werden die Transformation States generiert. Diese bestehen aus folgenden Informationen:

- **Automatisch entfernte Features**: Namen der aufgrund von Cardinality oder Missing Rate entfernten Features.
- **Manuell entfernte Features**: Namen der anhand der Benutzerangabe **manual_columns_to_drop** entfernten Features.
- **Semantische Datentypen und Imputed Values**: Die ermittelten semantischen Datentypen sowie die Imputed Values für jedes einzelne Feature.
- **Unique Values und One-Hot-Encoder**: Erweiterte Unique Values und trainierte One-Hot-Encoder für die mit **cols_for_onehot** ausgewählten Features.
- **Skalierer**: Trainierte Skalierer für die mit **cols_and_scalers** ausgewählten Features.

Anhand dieser Angaben lassen sich dann Validierungs- und Testdaten verarbeiten, ohne die Trainingsdaten direkt einbeziehen zu müssen.

2. Ausführung und Ergebnisse

Die Architektur der Pipeline ermöglicht es, das Training eines neuronalen Netzwerkes mit fünf Methodenaufrufen (inklusive des Konstruktors) durchzuführen. Auf die Erzeugung einer Instanz der Klasse `NeuralNetworkPipeline` mittels des gleichnamigen Konstruktors folgt das Laden des zu trainierenden neuronalen Netzwerkes mit **`set_model`**. Anschließend werden der Trainingsalgorithmus mit **`set_trainalgo`** sowie die Konfiguration für die Datenvorverarbeitung mit **`set_preprocessing_options`** festgelegt. Nun kann das Training des neuronalen Netzwerkes mit der Methode **`train`** gestartet werden. Dieser Prozess erfolgt auf Basis des ausgewählten Trainingsalgorithmus, der Early-Stopping-Strategy sowie der Vorverarbeitung der Daten anhand der festgelegten Konfiguration für die Datenvorverarbeitung. Mit der Methode **`show_trainchart`** kann sich der Benutzer den Trainingsverlauf als Diagramm anzeigen lassen. Für die spätere Wiederverwendung wird die Pipeline abschließend mit der Methode **`save_pipeline_in_joblib`** abgespeichert.

Um eine Vorhersage autark von den Trainingsdaten durchzuführen, lässt sich die zuvor gespeicherte Pipeline in einem neuen Skript mit dem Befehl **`load`** der Bibliothek `joblib` laden. Zur Vorhersage müssen dann nur noch die Testdaten an die Methode **`predict`** übergeben werden.

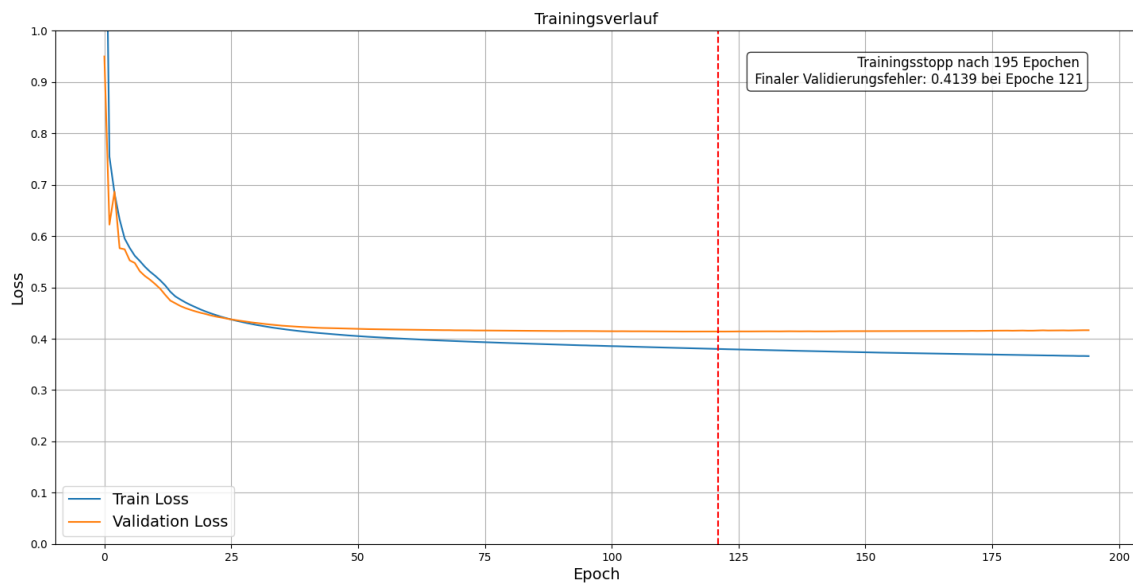
Zur Überprüfung der Pipeline wird im Folgenden ein Klassifikationsproblem gelöst. Dabei soll das Überleben der Passagiere der Titanic vorhergesagt werden, wofür der Kaggle-Titanic-Datensatz verwendet wird.

Entsprechend der Klassifikationsaufgabe erstellt man ein neuronales Netzwerk, das in der Ausgangsschicht die Sigmoid-Funktion zur Aktivierung nutzt.

Um das Modell zu trainieren, wird die Binary-Crossentropy als Verlustfunktion minimiert. Diese ist in den Optionen für den Trainingsalgorithmus hinterlegt.

Zur Vermeidung von Overfitting wird eine Early-Stopping-Strategy verwendet. Dieser Early-Stopper beendet das Training, falls der Validierungsfehler über einen Zeitraum von 80 Epochen hinweg eine Verbesserung von weniger als 0,0001 aufweist.

Mit diesen Einstellungen entsteht der folgende Trainingsverlauf:



Anfänglich sinken sowohl der Trainings- als auch der Validierungsfehler sehr stark. Mit steigender Epochenzahl schwächt sich dieser Abfall jedoch deutlich ab. Nach 25 Epochen nehmen beide Fehlerwerte nur noch geringfügig ab, wobei der Trainingsfehler stärker sinkt als der Validierungsfehler. Ab der 50. Epoche stagniert der Validierungsfehler nahezu, während der Trainingsfehler weiter langsam fällt. Da der Validierungsfehler über einen Zeitraum von 80 Epochen hinweg keine Verbesserung mehr zeigt, bricht der Early-Stopper das Training bei der Epoche 195 ab. Innerhalb dieser 80 Epochen wird bei Epoche 121 der geringste Validierungsfehler mit 41,39 % erreicht. Deshalb werden die Gewichte des trainierten neuronalen Netzwerkes auf die Werte bei Epoche 121 zurückgesetzt. Somit liefert das Training ein Modell mit einem Validierungsfehler von 41,39 %.

Insgesamt wurde hier ein typischer Trainingsverlauf erreicht. Beide Fehler sinken wie erwartet bei steigender Epochenzahl. Der Validierungsfehler sinkt nach einer bestimmten Anzahl von Epochen fast gar nicht mehr, während der Trainingsfehler weiter langsam sinkt. Der Early-Stopper stoppt das Training bei stagnierendem Validierungsfehler, um das Training vor dem Ansteigen des Validierungsfehlers zu beenden. Dies zeigt ein gelungenes Training des neuronalen Netzwerkes.

Des Weiteren ließ sich die Pipeline, mit dem trainierten neuronalen Netzwerk und den dazugehörigen Transformation States, abspeichern und in einem neuen Skript für Vorhersagen laden. Dort konnte erfolgreich eine Vorhersage mit den Testdaten durchgeführt werden, ohne dafür auf die Trainingsdaten zuzugreifen. Die Autarkie der Pipeline von den Trainingsdaten ist daher gewährleistet.